SAD

FACTS Case Management System

System Architecture & Design Document

Baaseteen / SHND — Internal Technical Reference

DOCUMENT VERSION	SYSTEM VERSION	DATE
3.7	FACTS v3.7	April 23, 2025
STATUS	**CLASSIFICATION**	**ORGANIZATION**
Final Draft	Internal / Restricted	Baaseteen / SHND

• ARCHITECTURE REFERENCE — V3.7

Table of Contents

FACTS System Architecture & Design Document — Version 3.7

DOCUMENT INFO

1. Document Overview
   - 1.1 Purpose
   - 1.2 Scope
   - 1.3 Assumptions & Constraints

OVERVIEW

2. System Overview
   - 2.1 System Description
   - 2.2 System Context
   - 2.3 Stakeholders
3. Architecture Goals & Principles
   - 3.1 Goals
   - 3.2 Principles

4 · ARCHITECTURE VIEWS

4.1 Logical View
   - 4.1.1 Backend Module Map
   - 4.1.2 Frontend Module Map
   - 4.1.3 Approval Workflow
4.2 Physical / Deployment View
   - 4.2.1 Deployment Architecture
   - 4.2.2 Nginx Configuration
4.3 Process View
   - 4.3.1 Request Lifecycle
   - 4.3.2 Concurrency & Connections
   - 4.3.3 Error Handling
4.4 Development View
   - 4.4.1 Technology Stack
   - 4.4.2 Project Structure
   - 4.4.3 Coding Conventions

5–6 · DATA & SECURITY

5. Data Architecture
   - 5.1 Entity Groups
   - 5.2 Key Relationships
   - 5.3 Schema Design Patterns
6. Security Architecture
   - 6.1 Authentication & Authorization
   - 6.2 Data Security
   - 6.3 Network Security

7–9 · INTEGRATION & RESILIENCE

7. Integration & Interfaces
   - 7.1 External Integrations (ITS, SMTP)
   - 7.2 Internal API Design Conventions
8. Performance & Scalability
   - 8.1 Performance Targets
   - 8.2 Scalability Design
9. Availability & Resilience
   - 9.1 Redundancy & Failover
   - 9.2 Backup & Disaster Recovery

10–12 · OPERATIONS

10. Deployment & Operations
   - 10.1 Deployment Environments
   - 10.2 Production Deployment Steps
   - 10.3 Monitoring & Alerting
   - 10.4 Maintenance Schedule
12. Technical Infrastructure
   - 12.1 Hosting Strategy
   - 12.2 Infrastructure Configuration
   - 12.3 Monitoring & Logging
   - 12.4 CI/CD Pipeline

APPENDICES

Appendix A — Glossary
Appendix B — API Endpoint Index
   - Authentication & Users
   - Cases & Workflow
   - Identification, Applicants & Forms
   - Admin, Master Data & Reports
Appendix C — DB Table Reference
Appendix D — Document Change Log

DOCUMENT METADATA

System: FACTS File & Case Tracking System
Organizations: Baaseteen & SHND
Version: SAD v3.7 / System v3.7
Standard: 4+1 Architectural View Model
Backend: 150+ endpoints, 25 modules
Database: 60+ tables (MySQL 8.0)

System Architecture and Design Document

FACTS — File and Case Tracking System

Document Version: 3.7
System Version: FACTS v3.7
Date: April 23, 2025
Prepared By: FACTS System Team
Status: Final Draft
Classification: Internal / Restricted
Organizations: Baaseteen & SHND

1. Document Overview

1.1 Purpose

This System Architecture and Design Document (SAD) is the authoritative technical reference for the FACTS (File and Case Tracking System). It is intended to communicate structural decisions, design rationale, and operational guidelines to engineers, architects, administrators, and technical stakeholders involved in building, deploying, maintaining, or extending the system.

The document covers end-to-end architecture including logical decomposition into modules, physical deployment topology, data modeling, security controls, integration patterns, performance targets, and operational procedures.

1.2 Scope

This document covers the complete FACTS platform deployed for the Baaseteen and SHND welfare organizations. System scope includes:

- Backend REST API (Node.js / Express.js) — 150+ endpoints across 25 route modules
- Frontend Single-Page Application (React 18) — 31 page components with role-adaptive UI
- MySQL 8.0 relational database — 60+ normalized tables
- File storage for case attachments, digital signatures, photos, and generated PDFs
- Integration with the external ITS (Institutional Information System) API
- Optional SMTP email notification pipeline

Out of scope: Third-party ITS platform internals, SMTP relay infrastructure, and any future mobile application layer.

1.3 Assumptions and Constraints

ASSUMPTIONS

- Node.js v16+ and MySQL v8.0+ available in target environment
- External ITS API accessible over HTTPS with valid credentials
- Users access via modern web browser (Chrome, Firefox, Edge)
- File uploads do not exceed 10 MB per file
- Concurrent users per deployment ≤ 500

CONSTRAINTS

- Jamiat/Jamaat organizational scoping required for data isolation
- JWT tokens expire after 24 hours; no refresh token in v1
- PDF generation requires Puppeteer (Chromium binary dependency)
- MySQL connection pool capped at 10 concurrent connections per instance
- Base64-encoded photos stored directly in DB (no object storage in v1)

2. System Overview

2.1 System Description

FACTS is a comprehensive welfare case management platform designed to digitize and govern the full lifecycle of financial assistance cases for Baaseteen and SHND organizations. It replaces paper-based counseling forms, manual approval chains, and fragmented tracking spreadsheets with a unified, role-aware digital workflow.

A case begins with an eligibility screening (Case Identification), progresses through a structured counseling phase, then traverses a multi-tier approval pipeline — Welfare Review → ZI Approval → KG Review → Operations Lead → Cover Letter Generation → Executive Approval → Manzoori Sanction — before reaching Financial Disbursement and periodic Fund Utilization tracking, ultimately closing with documented evidence of aid completion.

2.2 System Context

EXTERNAL: ITS API

Institutional Information System. Provides applicant biographical data, photos, Jamiat/Jamaat affiliations by ITS number. Read-only HTTPS REST integration.

FACTS PLATFORM

Core system: Express.js API + React SPA + MySQL. Manages all case data, workflows, approvals, documents, notifications, and reports.

EXTERNAL: SMTP RELAY

Optional email delivery (Gmail SMTP or equivalent). Receives outbound notification payloads from the FACTS notification service.

User Roles: Super Admin, Admin, DCM (Deputy Counseling Manager), Counselor, Welfare Reviewer, ZI (Zonal Incharge), KG Reviewer, Operations Lead, Executive Management, Finance Officer, Report Viewer

2.3 Stakeholders

STAKEHOLDER	ROLE	SYSTEM INTERACTION
Super Admin	Platform owner	Full system access, role/permission configuration, master data management
Admin	Operational administrator	User management, case oversight, reporting
DCM	Case manager	Case assignment, counseling form oversight, cover letter submission
Counselor	Field counselor	Fill counseling forms, capture signatures, attach documents
Welfare Reviewer	First-tier reviewer	Review cases, complete welfare checklist, approve/reject/rework
ZI / KG / Ops Lead	Mid-tier approvers	Review and approve/reject cases at their assigned workflow stage
Executive Management	Senior approver	Multi-level executive approval hierarchy (up to 5 levels)
Finance Officer	Disbursement manager	Confirm disbursements, manage payment schedules
IT/DevOps Team	Infrastructure owner	Deployment, monitoring, backup, security patching

3. Architecture Goals and Principles

3.1 Goals

Correctness & Auditability
Every state transition, approval, and data edit is logged. Status history, workflow comments, and income audit logs provide a complete audit trail.

Role-Based Access Control
All resources protected by a fine-grained permission model (resource + action). Organizational scoping (Jamiat/Jamaat) further restricts data visibility.

Workflow Configurability
Approval stages, role assignments, SLA thresholds, and checklist items are database-driven — configurable without code changes.

Performance & Responsiveness
API responses within 300ms for paginated list endpoints, 500ms for complex aggregations under normal load.

Data Integrity
Foreign-key constraints, parameterized queries, and transactional operations protect relational integrity across 60+ tables.

3.2 Principles

Separation of Concerns
API routes, business logic, and database access are structurally separated. Frontend pages, components, and contexts independently organized.

Fail Safely
Errors return structured JSON with meaningful status codes. Unhandled exceptions caught by global middleware without exposing stack traces to clients.

Least Privilege
Tokens carry minimal claims. Every request revalidates permission and organizational scope against the live database.

Minimal External Dependencies
System avoids cloud-vendor lock-in. Core functionality runs on any Linux server with Node.js and MySQL.

Progressive Enhancement
System degrades gracefully if optional services (SMTP, Puppeteer) are unavailable; core case management remains functional.

4.1 Logical View — Modules & Component Interaction

4.1.1 Backend Module Map

The backend is organized into 25 route modules, each encapsulating a bounded domain. All follow the pattern: `express.Router → middleware chain → DB query → JSON response` .

MODULE	ROUTE PREFIX	CORE RESPONSIBILITY	KEY DEPENDENCIES
Auth	`/api/auth`	Login, JWT issuance, registration, profile, password management	bcryptjs, jsonwebtoken
Users	`/api/users`	User CRUD, role assignment, ITS contact sync, Excel import/export	xlsx, ITS API
Cases	`/api/cases`	Case CRUD, all workflow action handlers, comments, payment schedules, closure	notificationService, slaCalculator
Case Identifications	`/api/case-identifications`	Pre-assessment eligibility screening, income audit logging	activeCasePerIts
Applicants	`/api/applicants`	Applicant profiles, ITS lookup, bulk import/fetch	ITS API, xlsx
Counseling Forms	`/api/counseling-forms`	Multi-section form data management (7 sub-tables)	counseling_form_stage_permissions
Cover Letters	`/api/cover-letters`	PDF cover letter generation and download	Puppeteer
Cover Letter Forms	`/api/cover-letter-forms`	Cover letter form data and submission workflow	Cases, notificationService
Attachments	`/api/attachments`	Multipart file upload (Multer), download, delete, statistics	Multer, fs
Workflow Stages	`/api/workflow-stages`	Stage definitions, role/user assignments, SLA config, ordering	roles, users
Welfare Checklist	`/api/welfare-checklist`	Checklist categories, items, per-case responses, completion tracking	cases
Fund Utilization	`/api/cases/:id/fund-utilization`	Quarterly reassessment reports, photo/file uploads	fundUtilizationMaster
Fund Utilization Master	`/api/fund-utilization-master`	Dynamic form builder — sections and questions configuration	—
Dashboard	`/api/dashboard`	Role-filtered aggregate statistics, pipeline data, recent activities	cases, users
Notifications	`/api/notifications`	In-app notifications, read state, SLA check trigger	notificationService
Roles / Permissions	`/api/roles` , `/api/permissions`	RBAC role and permission management, access verification	role_permissions, user_roles
Quarterly Report PDF	`/api/cases/:id/quarterly-report`	PDF generation for fund utilization quarterly reports	PDFKit, quarterlyReportPdfData
Master Data	`/api/jamiat` , `/api/jamaat` , etc.	Reference data CRUD: Jamiat, Jamaat, Case Types, Relations, Education Levels, Occupations, Executive Levels	—

4.1.2 Frontend Module Map

PAGE MODULES (31)

Login, Dashboard, Cases, CaseDetails, CaseIdentification, Applicants, Users, RoleManagement, JamiatMaster, WorkflowStages, WelfareChecklist, FundUtilization, PaymentSchedulePage, Reports, Profile, Notifications, and 15 master-data pages.

DOMAIN COMPONENTS (10)

CounselingForm (multi-section), CoverLetterForm, FundUtilization components, PaymentSchedule, WelfareChecklistForm, BusinessAssets, FileUpload, SignaturePad, CaseSLAStatus, WorkflowComments.

UI LIBRARY (16)

Button, Card, Modal, Table, Alert, Badge, Input, Select, SearchableSelect, MultiSelect, Pagination, Toast, Tabs, Spinner, WorkflowProgress — all custom-built with Tailwind CSS.

4.1.3 Approval Workflow Data Flow

At each approval stage, cases may be rejected or returned for rework to a prior stage. In-app notifications are dispatched on every transition.

4.2 Physical View — Deployment & Network Design

4.2.1 Deployment Architecture

COMPONENT PLACEMENT

Web Server (Nginx)
Serves React build (static files), terminates SSL, reverse-proxies `/api` and `/uploads` to Node.js on port 5000.

Application Server (Node.js)
Single Express process managed by PM2. Listens on port 5000 (localhost only). Cluster mode available for multi-core servers.

Database Server (MySQL 8.0)
Same host for small deployments or dedicated VM. Uses InnoDB engine; connection pool size: 10.

File Storage
Local `./uploads/` directory. Served via Express static middleware at `/uploads/*` .

NETWORK TOPOLOGY

Public Ports
443 (HTTPS), 80 (HTTP → HTTPS redirect). All other ports closed by firewall.

Internal Ports
5000 (Node.js API, localhost only), 3306 (MySQL, localhost only).

SSL Termination
Nginx handles TLS 1.2+. Certificates via Let's Encrypt (certbot) or organization CA.

ITS API Egress
Outbound HTTPS to ITS endpoint. Must be whitelisted in egress firewall rules.

4.2.2 Nginx Configuration Summary

```
server {
  listen 443 ssl http2;
  server_name your-domain.com;

  # Frontend — serve React build
  location / {
    root /var/www/FACTS/frontend/build;
    try_files $uri $uri/ /index.html;   # SPA fallback
  }

  # Backend API proxy
  location /api {
    proxy_pass http://localhost:5000;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header Host $host;
  }

  # File uploads passthrough
  location /uploads {
    proxy_pass http://localhost:5000/uploads;
  }

  client_max_body_size 10M;
}
```

4.3 Process View — Operations & Performance

4.3.1 Request Lifecycle

Every HTTP request passes through the following Express middleware chain:

1. **CORS** — Validates `Origin` against `FRONTEND_URL`
2. **Helmet** — Sets security-related HTTP response headers
3. **express-rate-limit** — Per-IP rate limit on `/api/auth/login`
4. **express.json()** — Parses JSON request bodies
5. **auth middleware** — Verifies JWT, loads user from DB, checks `is_active`
6. **permission middleware** — Validates `resource:action` and Jamiat/Jamaat scope
7. **Route handler** — Business logic and DB queries
8. **Error handler** — Catches thrown errors, returns structured JSON

4.3.2 Concurrency & Connection Management

> **MYSQL CONNECTION POOL**
>
> Configured with a maximum of 10 simultaneous connections. Requests queue when the pool is exhausted. For higher concurrency, increase the pool size in `config/database.js` and align MySQL server's `max_connections` accordingly.

Node.js handles requests asynchronously via the event loop. All database operations use async/await with `mysql2/promise`. File I/O (Multer uploads, PDF generation) is non-blocking via Node's libuv thread pool.

4.3.3 Error Handling & Fault Tolerance

ERROR CLASS	HANDLING STRATEGY	CLIENT RESPONSE
Validation errors	Joi schema validation before DB access	400 + field-level error message
Auth failures	JWT verify + active user DB check	401 / 403 JSON error
DB constraint violations	mysql2 error code inspection	409 / 400 JSON error
File upload errors	Multer error handler middleware	400 + Multer error message
PDF generation failure	try/catch around Puppeteer/PDFKit	500 + "PDF generation failed"
External ITS API timeout	Axios timeout + catch, fallback to DB	503 or cached DB data
Unhandled exceptions	Global Express error middleware	500 + generic message (no stack trace)

4.4 Development View — Code & Environment

4.4.1 Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Backend Runtime	Node.js	v16+	JavaScript runtime for API server
Backend Framework	Express.js	v4.18	HTTP routing, middleware, request handling
Database	MySQL (InnoDB)	v8.0+	Relational data storage with FK constraints
DB Client	mysql2/promise	—	Async MySQL driver with connection pooling
Authentication	jsonwebtoken + bcryptjs	v9 / v2	JWT issuance; password hashing (bcrypt, 10 rounds)
Frontend Framework	React	v18.2	Component-based SPA with hooks
Server State	React Query	v3.39	API data caching, mutations, background refetch
Forms	React Hook Form	v7.48	Performant form state and validation
Styling	Tailwind CSS	v3.4	Utility-first CSS framework
HTTP Client	Axios	v1.6	API calls from frontend with interceptors
Routing	React Router	v6	SPA client-side routing with protected routes
PDF Generation	PDFKit + Puppeteer	v0.14 / v24	PDFKit for data reports; Puppeteer for HTML→PDF
File Uploads	Multer	v1.4	Multipart form data processing
Excel I/O	xlsx (SheetJS)	v0.18	Bulk import/export for users, applicants, Jamiat
Validation	Joi	v17.11	Schema validation for request bodies
Security	Helmet, express-rate-limit	—	HTTP header hardening, login rate limiting
Email	Nodemailer	v6.9	Optional SMTP notification dispatch
Charts	Recharts	v2.8	Dashboard analytics visualizations
Drag & Drop	@dnd-kit	—	Workflow stage reordering UI

4.4.2 Project Structure

```
FACTS/
├── backend/
│   ├── config/database.js          # MySQL connection pool (max: 10)
│   ├── middleware/
│   │   ├── auth.js                 # JWT verification + scope injection
│   │   └── permissionMiddleware.js # resource:action permission checks
│   ├── routes/                     # 25 route modules (one domain per file)
│   ├── services/
│   │   ├── emailService.js         # SMTP email dispatch via Nodemailer
│   │   └── notificationService.js  # In-app notification creation
│   ├── utils/
│   │   ├── activeCasePerIts.js     # Enforce 1 active case per ITS number
│   │   ├── slaCalculator.js        # SLA deadline computation
│   │   └── quarterlyReportPdfKit.js# PDFKit quarterly report builder
│   └── index.js                    # Express server entry point
│
└── frontend/src/
    ├── components/
    │   ├── Layout/                 # App shell (sidebar, header, nav)
    │   ├── [Domain components]     # CounselingForm, PaymentSchedule…
    │   └── ui/                     # 16 reusable UI primitives
    ├── pages/                      # 31 full-page views
    ├── contexts/AuthContext.js     # Global auth state management
    └── utils/permissionUtils.js    # usePermission hook
```

4.4.3 Coding Conventions

- All backend queries use parameterized statements via mysql2 — no dynamic SQL string construction
- Frontend components use functional style exclusively (no class components); state managed via hooks
- API responses follow consistent envelope: success returns data payload, errors return `{ "error": "message" }`
- Environment-specific config lives exclusively in `.env`; never hardcoded in source
- Route files are single-concern; shared logic lives in `utils/` or `services/`

5. Data Architecture and Design

5.1 Entity Groups

IDENTITY & ACCESS

`users` , `roles` , `user_roles` , `role_permissions` , `counseling_form_stage_permissions`

CASE CORE

`cases` , `applicants` , `case_types` , `statuses` , `case_identifications` , `case_identification_income_logs`

COUNSELING FORM

`counseling_forms` , `personal_details` , `family_details` , `family_members` , `assessment` , `assessment_products_services` , `financial_assistance` (+ 5 child tables), `economic_growth` , `economic_growth_projections` , `declaration` , `attachments`

WORKFLOW & APPROVAL

`workflow_stages` , `workflow_stage_roles` , `workflow_stage_users` , `workflow_comments` , `status_history` , `executive_levels` , `cover_letters` , `cover_letter_forms`

FINANCE & FUND

`payment_schedules` , `payment_schedule_repayments` , `business_assets` , `fund_utilization_reports` , `fund_utilization_photos` , `fund_utilization_files` , `fund_utilization_sections` , `fund_utilization_questions`

ORGANIZATION & SUPPORT

`jamiat` , `jamaat` , `relations` , `education_levels` , `occupations` , `notifications` , `case_attachments` , `case_comments` , `case_closures` , `welfare_checklist_categories/items/responses` , `system_settings`

5.2 Key Relationships

PARENT TABLE	CHILD TABLES	RELATIONSHIP
`cases`	`counseling_forms` , `cover_letter_forms` , `payment_schedules` , `case_attachments` , `status_history` , `fund_utilization_reports` , `case_closures`	One case → many child records
`counseling_forms`	`personal_details` , `family_details` , `assessment` , `financial_assistance` , `economic_growth` , `declaration` , `attachments`	One form → one record per section (FK IDs)
`financial_assistance`	`fa_timeline` , `fa_action_plan` , `fa_mentors` , `fa_qh_repayment_schedule` , `fa_timeline_assistance`	One FA record → many detail rows
`workflow_stages`	`workflow_stage_roles` , `workflow_stage_users`	Stage → many role/user permission assignments
`jamiat`	`jamaat`	One Jamiat → many Jamaat
`users`	`user_roles`	Many-to-many via user_roles (with scope columns)
`fund_utilization_sections`	`fund_utilization_questions`	Section → many questions (hierarchical via parent_question_id)

5.3 Schema Design Patterns

- **Soft deletes:** Roles, workflow stages, and checklist items use `is_active` boolean to preserve referential integrity and audit history rather than physical deletion.
- **JSON columns:** `workflow_history` (cases), `associated_statuses` (workflow_stages), `responses_json` (fund_utilization_reports) store variable-structure data without schema migrations.
- **Base64 photos:** Stored as `longtext` in database (v1 constraint). Recommendation: migrate to object storage path reference in v2 for scale.
- **Audit logging:** `case_identification_income_logs` captures before/after values for income edits. `status_history` captures every case status transition with actor and timestamp.
- **SLA fields:** `workflow_stages.sla_value` , `sla_unit` , `sla_warning_value` , `sla_warning_unit` drive the automated SLA calculator per stage.
- **Dynamic forms:** `fund_utilization_sections` and `fund_utilization_questions` allow admin-configurable form structure without code changes.

6. Security Architecture and Design

6.1 Authentication & Authorization

AUTHENTICATION FLOW

1. User submits credentials to `POST /api/auth/login`
2. bcryptjs compares password against stored hash (10 salt rounds)
3. On success: signed JWT issued with `{ userId, role, iat, exp }`
4. Token lifetime: 24 hours (configurable via `JWT_EXPIRES_IN`)
5. Client sends `Authorization: Bearer <token>` on every request
6. Auth middleware reloads user from DB and checks `is_active = 1` on every request

AUTHORIZATION LAYERS

Layer 1 — RBAC: Permissions stored as `resource:action` pairs in `role_permissions`. Effective permissions resolved from all assigned roles and injected into `req.user.permissions`.

Layer 2 — Organizational Scope: `user_roles` stores `jamiat_ids` and `jamaat_ids` per role. Middleware filters case queries to the user's authorized organizations.

Layer 3 — Stage Permissions: `counseling_form_stage_permissions` controls per-role read/update access at the form section level.

6.2 Data Security

CONTROL	IMPLEMENTATION
Password Storage	bcryptjs, 10 salt rounds — no plaintext passwords ever stored
Transport Encryption	TLS 1.2+ enforced at Nginx layer for all client traffic
Data at Rest	MySQL tablespace encryption (AES-256) recommended for production deployments
SQL Injection Prevention	100% parameterized queries via mysql2 prepared statements
Input Validation	Joi schema validation on all POST/PUT request bodies
File Upload Security	Multer enforces MIME type whitelist and 10 MB per-file size limit
Sensitive Configuration	DB credentials, JWT secret, SMTP password in `.env` — never committed to source control

6.3 Network Security

- **Helmet.js** sets Content-Security-Policy, X-Frame-Options, HSTS, and other protective HTTP headers on all API responses
- **CORS** restricts `Access-Control-Allow-Origin` to the configured `FRONTEND_URL` only
- **Rate Limiting** on `/api/auth/login` via express-rate-limit; trusted IPs whitelisted via `LOGIN_RATE_LIMIT_SKIP_IPS`
- **Firewall:** Only ports 80, 443, and 22 (SSH) externally accessible. MySQL (3306) and Node.js (5000) must be bound to localhost only
- **Egress:** ITS API calls are outbound only; credentials stored in environment variables

7. Integration and Interface Design

7.1 External System Integrations

ITS (Institutional Information System) API

> **INTEGRATION PATTERN: OUTBOUND REST (HTTPS GET), READ-ONLY, ON-DEMAND**
> FACTS calls the ITS API to fetch applicant biographical data by ITS number. Results are cached in the `applicants` table. If the ITS API is unavailable, FACTS falls back to the cached DB record or returns a 503.

TRIGGER	FACTS ENDPOINT	FALLBACK
New case identification lookup	`GET /api/applicants/lookup/:itsNumber`	Returns DB record if exists
Bulk applicant ITS fetch	`POST /api/applicants/bulk-fetch`	Marks failed records with error summary
User contact sync by role	`POST /api/users/fetch-contact-from-api`	Reports per-user error list
Case ID creation (public endpoint)	`GET /api/applicants/fetch-from-api/:itsNumber`	503 if ITS unreachable

SMTP Email (Optional)

The `emailService.js` module uses Nodemailer with SMTP transport. Triggered by `notificationService.js` on case status transitions. If `EMAIL_HOST` is not configured, the system operates without email delivery — all notifications remain in-app only.

7.2 Internal API Design Conventions

CONVENTION	DETAIL
Base URL	`https://domain.com/api`
Authentication	`Authorization: Bearer <JWT>` header on all protected endpoints
Data Format	JSON request/response bodies; `Content-Type: application/json`
File Uploads	`multipart/form-data` for all file upload endpoints
Pagination	Query params `page` (default 1), `limit` (default 10–20). Response includes `pagination.total` and `pages`.
Success codes	200 (OK), 201 (Created)
Error codes	400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), 409 (Conflict), 500 (Server Error)
Error envelope	`{ "error": "descriptive message" }`

8. Performance, Scalability, and Optimization

8.1 Performance Targets

ENDPOINT TYPE	TARGET RESPONSE TIME	NOTES
Authentication (login)	< 200 ms	bcrypt comparison ~100ms at 10 rounds
List endpoints (paginated)	< 300 ms	Cases, users, notifications with default limit=10
Single record fetch	< 150 ms	Case detail, user by ID
Workflow actions	< 400 ms	Status update + notification creation
PDF generation (cover letter)	< 3 s	Puppeteer HTML→PDF; Chromium startup overhead
Excel import (100 rows)	< 2 s	SheetJS parse + batch DB inserts
Dashboard overview	< 500 ms	Multiple aggregate COUNT queries

8.2 Scalability Design

VERTICAL SCALING

Increase server CPU/RAM. Node.js benefits from more heap for Puppeteer. MySQL benefits significantly from a larger InnoDB buffer pool (target 70–80% of available RAM).

HORIZONTAL SCALING

PM2 cluster mode: `pm2 start index.js -i max` spawns one process per CPU core. JWT is stateless so no sticky sessions needed. Requires shared MySQL and shared file storage (NFS or object store).

DATABASE OPTIMIZATION

Key indexes on: `cases.status`, `cases.jamiat_id`, `cases.applicant_id`, `applicants.its_number`, `notifications.user_id + is_read`. All list endpoints paginated — never unbounded SELECT.

CACHING STRATEGY

Frontend: React Query caches server data with configurable `staleTime`. Master data (case types, Jamiat) refetched infrequently. Backend: No server-side cache in v1; Redis can be added in v2 for session/rate-limit state.

BASE64 PHOTO STORAGE WARNING

Storing photos as Base64 longtext inflates row sizes significantly. For deployments with >1,000 applicants, migrate photos to file system paths or S3-compatible object storage and store only the URL in the DB.

9. System Availability and Resilience Design

9.1 Redundancy & Failover

COMPONENT	REDUNDANCY STRATEGY	RTO TARGET
Application Server	PM2 auto-restart on crash or systemd `Restart=on-failure`	< 30 seconds
Database	MySQL primary + read replica replication; manual failover promotion	< 5 minutes
Nginx	systemd auto-restart; keepalived for dual-Nginx HA if required	< 30 seconds
File Storage	RAID or replicated NFS for `/uploads` ; daily rsync to backup location	< 1 hour (restore)

9.2 Backup & Disaster Recovery

RECOMMENDED BACKUP SCHEDULE

Daily full MySQL dump with 30-day retention. Weekly off-site copy. Uploads directory synced daily to secondary storage via rsync or cloud backup agent.

```
# Daily MySQL backup cron (example — runs at 2 AM)
0 2 * * * mysqldump -u root -p baaseteen_cms \
  --single-transaction --quick \
  | gzip > /backups/facts_$(date +%F).sql.gz

# Retain last 30 backups, delete older
find /backups -name "facts_*.sql.gz" -mtime +30 -delete
```

Recovery procedure: restore from latest dump (`mysql baaseteen_cms < backup.sql`), restore uploads directory, restart Node.js service. Full recovery test recommended quarterly.

10. Deployment, Operations, and Maintenance

10.1 Deployment Environments

ENVIRONMENT	PURPOSE	CONFIGURATION NOTES
Development	Local engineer workstation	`NODE_ENV=development` ; React dev server (port 3000); backend nodemon (port 5000); local MySQL
Staging	Pre-production validation	Mirrors production config; anonymized data; all releases tested here before production
Production	Live system	`NODE_ENV=production` ; React build served by Nginx; PM2/systemd; SSL; strong JWT secret

10.2 Production Deployment Steps

1. Build React frontend: `cd frontend && npm run build`
2. Copy build to web root: `/var/www/FACTS/frontend/build/`
3. Pull latest backend, install deps: `npm install --production`
4. Apply DB migrations: `mysql baaseteen_cms < migrations/<file>.sql`
5. Restart application: `pm2 restart facts-backend`
6. Verify health: `curl https://domain.com/health`

10.3 Monitoring & Alerting

APPLICATION MONITORING

- PM2 built-in: `pm2 monit` for real-time process stats
- Health endpoint: `GET /health` returns server status
- Error logging: stdout/stderr via PM2 log files
- Recommended: Grafana + Prometheus Node.js exporter for metrics dashboards

SLA MONITORING

- Built-in SLA calculator computes `on_time` / `at_risk` / `breached` per case stage
- Admin trigger: `POST /api/notifications/check-sla` creates alerts
- Recommended: schedule SLA check cron every 4 hours

10.4 Maintenance Schedule

- **Monthly:** `npm audit` on backend and frontend; apply security patches; `OPTIMIZE TABLE` on high-write tables (notifications, status_history)
- **Weekly:** Review PM2 error logs; check backup completion; verify SSL cert expiry dates
- **Quarterly:** Full disaster recovery test; review and rotate JWT secret; test SMTP delivery

12. Technical Architecture for Hosting, Development & Infrastructure

12.1 Hosting Strategy

FACTS is designed as a self-hosted, on-premises or private-cloud application. This ensures full data sovereignty for sensitive welfare case information and avoids recurring cloud vendor costs for small-to-medium scale deployments.

RESOURCE	MINIMUM SPECIFICATION	RECOMMENDED
CPU	2 vCPU	4 vCPU (Puppeteer PDF generation is CPU-intensive)
RAM	4 GB	8 GB (InnoDB buffer pool + Chromium memory)
Storage	50 GB SSD	200 GB SSD (attachments + DB growth over time)
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS (LTS support until 2027)
Network	100 Mbps	1 Gbps for optimal file upload performance

12.2 Infrastructure Configuration

Compute

- PM2 cluster mode: `pm2 start index.js -i max` — utilizes all CPU cores
- Puppeteer requires Chromium: `apt-get install chromium-browser`
- Set `NODE_OPTIONS=--max-old-space-size=2048` for large PDF generation workloads

Database

- Enable `innodb_file_per_table=1` for easier per-table management
- Set `innodb_buffer_pool_size` to 60–70% of server RAM
- Enable binary logging (`log_bin`) for point-in-time recovery
- Create a dedicated application DB user with minimum privileges: SELECT, INSERT, UPDATE, DELETE on FACTS DB only

Storage

- Upload directory: `/var/www/FACTS/backend/uploads/` must be writable by the Node.js process user
- Mount uploads on a separate logical volume to prevent disk-full events from crashing the OS

12.3 Monitoring and Logging

```
# PM2 log paths
~/.pm2/logs/facts-backend-out.log    # stdout
~/.pm2/logs/facts-backend-error.log  # stderr

# Nginx logs
/var/log/nginx/access.log
/var/log/nginx/error.log

# MySQL slow query log (add to /etc/mysql/my.cnf)
slow_query_log = 1
slow_query_log_file = /var/log/mysql/slow.log
long_query_time = 1    # log queries over 1 second
```

12.4 Development & CI/CD Pipeline

> **RECOMMENDED PIPELINE STAGES**
>
> (1) Install dependencies → (2) Backend unit tests → (3) Frontend lint + build → (4) Deploy to staging → (5) Smoke tests → (6) Manual approval → (7) Deploy to production

STAGE	TOOL	ACTION
Version Control	Git (GitHub / GitLab)	Feature branches → PR review → merge to main
CI — Test	GitHub Actions / GitLab CI	`npm test` for backend and frontend unit tests
CI — Build	Same CI runner	`cd frontend && npm run build`
CD — Staging	SSH deploy / rsync action	Deploy to staging, run DB migrations, restart PM2
CD — Production	Manual trigger (protected branch)	Same steps as staging after smoke test pass
Secrets Management	GitHub Secrets / GitLab Variables	JWT_SECRET, DB password, SSH keys — never in repository

Appendix A — Glossary

TERM	DEFINITION
FACTS	File and Case Tracking System — the platform documented in this SAD
ITS	Institutional Information System — external platform providing member data by ITS number
ITS Number	8-digit unique identifier assigned to each Dawoodi Bohra community member
DCM	Deputy Counseling Manager — primary case manager responsible for case assignment and counseling oversight
ZI	Zonal Incharge — regional supervisor performing first-tier case approval after welfare review
KG Review	Second approval tier after ZI Approval in the workflow pipeline
Jamiat	Top-level organizational unit (regional organization) in the Baaseteen/SHND structure
Jamaat	Sub-organizational unit (local congregation) belonging to a Jamiat
Mauze	Geographic locality designation used within Jamiat/Jamaat context
Manzoori	Final sanction/approval stage before financial disbursement
Enayat	Non-repayable grant component of financial assistance
Qardan Hasana (QH)	Interest-free repayable loan component of financial assistance
SLA	Service Level Agreement — maximum time a case should remain at a given workflow stage
RBAC	Role-Based Access Control — permission model assigning rights by user role
JWT	JSON Web Token — stateless authentication token signed with HMAC-SHA256
SAD	System Architecture and Design document (this document)
PM2	Production Process Manager for Node.js — manages restart, clustering, and logging
SPA	Single-Page Application — the React frontend that loads once and updates dynamically
Baaseteen	Primary welfare organization served by FACTS
SHND	Social Help and Development — secondary organization served by FACTS

Appendix B — API Endpoint Index

Summary of all 150+ API endpoints grouped by module. Full request/response schemas are in the separate *FACTS API Schema Reference* document.

Authentication & Users

METHOD	ENDPOINT	DESCRIPTION	AUTH REQUIRED
POST	`/api/auth/login`	Login, returns JWT token	No
POST	`/api/auth/register`	Create user (Admin only)	Admin
GET	`/api/auth/profile`	Current user profile + permissions	Yes
PUT	`/api/auth/change-password`	Change own password	Yes
POST	`/api/auth/logout`	Invalidate session	Yes
GET	`/api/users`	List users (filtered, paginated)	users:read
POST	`/api/users`	Create user with role assignment	users:create
PUT	`/api/users/:id`	Update user details	users:update
DELETE	`/api/users/:id`	Delete user permanently	users:delete
POST	`/api/users/import-excel`	Bulk import users from Excel	users:create
POST	`/api/users/fetch-contact-from-api`	Sync contact from ITS API by role	users:update

Cases & Workflow

METHOD	ENDPOINT	DESCRIPTION	AUTH REQUIRED
GET	`/api/cases`	List cases with filters and pagination	cases:read
POST	`/api/cases`	Create new case	cases:create
GET	`/api/cases/:id`	Full case detail with history and comments	Case access
PUT	`/api/cases/:id`	Update case (assignment, status)	Case access
PUT	`/api/cases/:id/welfare-approve`	Welfare department approval	welfare role
PUT	`/api/cases/:id/welfare-reject`	Welfare rejection with comments	welfare role
PUT	`/api/cases/:id/zi-approve`	ZI approval, advance stage	ZI role
PUT	`/api/cases/:id/zi-reject`	ZI rejection back to welfare	ZI role
PUT	`/api/cases/:id/executive-approve`	Executive multi-level approval	Executive role
PUT	`/api/cases/:id/executive-rework`	Executive rework to welfare	Executive role
PUT	`/api/cases/:id/workflow-action`	Generic approve/reject action	Stage role
POST	`/api/cases/:id/close`	Close case with closure documents	cases:close_case
GET	`/api/cases/:id/payment-schedule`	Get payment installment schedule	Case access
POST	`/api/cases/:id/payment-schedule`	Save payment schedule	Case access
POST	`/api/cases/:id/payment-schedule/:sid/confirm-disbursement`	Confirm payment disbursement	Case access

Case Identification, Applicants & Forms

METHOD	ENDPOINT	DESCRIPTION	AUTH REQUIRED
GET	`/api/case-identifications`	List identifications with filters	case_identification:read
POST	`/api/case-identifications`	Create new eligibility identification	case_identification:create
PUT	`/api/case-identifications/:id/review`	Mark eligible or ineligible (auto-creates case if eligible)	case_identification:approve

METHOD	ENDPOINT	DESCRIPTION	AUTH REQUIRED
PUT	`/api/case-identifications/:id/edit`	Edit assessment fields with audit log	case_identification:edit
GET	`/api/applicants/lookup/:itsNumber`	ITS number lookup (DB first, then external API)	Yes
POST	`/api/applicants/bulk-fetch`	Batch ITS API fetch for multiple applicants	applicants:update
GET	`/api/counseling-forms/case/:id`	Get full counseling form for a case	Case access
PUT	`/api/counseling-forms/:id/section/:section`	Update individual form section	Stage permission
PUT	`/api/counseling-forms/:id/complete`	Mark form as complete	Yes
POST	`/api/cover-letters/generate/:id`	Generate PDF cover letter from form data	Case access
PUT	`/api/cover-letter-forms/:id/submit`	Submit cover letter form for approval	cover_letter_forms:submit

Admin, Master Data & Reports

METHOD	ENDPOINT	DESCRIPTION	AUTH REQUIRED
GET	`/api/dashboard/overview`	Role-filtered dashboard statistics	Yes
GET	`/api/notifications`	User notifications (paginated)	Yes
PUT	`/api/notifications/mark-all-read`	Mark all notifications as read	Yes
POST	`/api/attachments/upload/:caseId`	Upload single file to case	Case access
POST	`/api/attachments/upload-multiple/:caseId`	Upload multiple files (max 10)	Case access
GET	`/api/roles`	List roles with permissions	Admin
GET	`/api/workflow-stages`	List configurable workflow stages	master:read
PUT	`/api/workflow-stages/reorder`	Reorder stages via drag-and-drop	master:update
GET	`/api/welfare-checklist/items/grouped`	Checklist items grouped by category	Yes
POST	`/api/welfare-checklist/responses/:caseId`	Submit checklist responses for a case	Yes
GET	`/api/cases/:id/fund-utilization`	List fund utilization reports for case	Yes
GET	`/api/cases/:id/quarterly-report/pdf`	Download quarterly report PDF	fund_utilization:read
GET	`/api/jamiat`	List all Jamiat organizations	Yes
GET	`/api/executive-levels`	Executive hierarchy level definitions	Yes

Appendix C — Database Table Reference

TABLE	PK	KEY COLUMNS	GROUP
users	id	username, email, role, executive_level, is_active, photo (longtext B64)	Identity
roles	id	name, display_name, permissions (JSON), is_system_role, is_active	Identity
user_roles	id	user_id, role_id, jamiat_ids, jamaat_ids, is_active, expires_at	Identity
role_permissions	id	role_id, resource, action	Identity
cases	id	case_number, applicant_id, status, roles (DCM user_id), current_workflow_stage_id, case_type_id, is_duplicate	Case Core
applicants	id	its_number (unique), full_name, gender, age, phone, jamiat_id, jamaat_id, photo (B64)	Case Core
case_identifications	id	its_number, eligible_in (case_type FK), status (pending/eligible/ineligible), individual_income, family_income, reviewed_by	Case Core
case_identification_income_logs	id	case_identification_id, old/new_individual_income, old/new_family_income, changed_by	Case Core
counseling_forms	id	case_id, is_complete, personal_details_id, family_details_id, assessment_id, financial_assistance_id, economic_growth_id, declaration_id, attachments_id	Counseling
personal_details	id	case_id, its_number, name, education, contact_number, present_occupation, residential_address	Counseling
family_details	id	case_id, income totals (monthly/yearly by type), expense totals, surplus/deficit, assets, liabilities_total	Counseling
assessment	id	case_id, background fields (education, work_experience, skills), proposed_business fields, counselor_assessment fields	Counseling
financial_assistance	id	case_id, assistance_required, self_funding, rahen_available, qh_fields (JSON), support_needed (JSON)	Counseling
economic_growth	id	case_id, revenue/expenses/profit projections (last_year + year1–5 per metric, 90+ decimal columns)	Counseling
declaration	id	case_id, applicant_its, signature_type (upload/draw), signature_file_path, counselor_signature, tr_committee fields	Counseling
workflow_stages	id	stage_name, stage_key, sort_order, sla_value, sla_unit, sla_warning_value, associated_statuses (JSON), is_active	Workflow
workflow_stage_roles	id	workflow_stage_id, role_id, can_approve, can_reject, can_review, can_view, can_edit, can_create_case, can_fill_case	Workflow
status_history	id	case_id, from_status, to_status, changed_by, comments, created_at	Workflow
cover_letter_forms	id	case_id, applicant/counselor fields, financial figures, requested/recommended/approved enayat+qardan, signature fields, is_complete, is_approved	Workflow
executive_levels	id	level_number, level_name, sort_order, is_active	Workflow
payment_schedules	id	case_id, payment_type (qardan_hasana/enayat), disbursement_year/month/amount, is_disbursed, disbursed_date, disbursed_by	Finance
business_assets	id	case_id, 8 asset types × 6 time periods (last_year + year1–5) = 48 decimal columns	Finance
fund_utilization_reports	id	case_id, quarter (q1–q4), year, responses_json (longtext)	Finance
fund_utilization_questions	id	section_id, label, field_type, is_required, parent_question_id (self-ref for conditionals), auto_fill_key	Finance
jamiat	id	jamiat_id (string code), name, is_active	Organization
jamaat	id	jamaat_id (string code), jamiat_id (FK), name, is_active	Organization
notifications	id	user_id, case_id, title, message, type (info/warning/success/error), is_read	Support
case_attachments	id	case_id, file_name, file_path, file_type, file_size, stage, uploaded_by	Support

TABLE	PK	KEY COLUMNS	GROUP
`case_closures`	id	case_id, reason, closed_by, created_at	Support
`welfare_checklist_items`	id	category_id, form_section, checklist_detail, is_compulsory, is_active	Support
`welfare_checklist_responses`	id	case_id, checklist_item_id, properly_filled (y/n), comments, filled_by	Support

Appendix D — Document Change Log

VERSION	DATE	AUTHOR	CHANGES
3.7	April 23, 2025	System Team	Initial release — complete architecture documentation for FACTS v3.7 covering all 12 SAD sections and 4 appendices

DOCUMENT MAINTENANCE POLICY

This document should be updated whenever significant architectural decisions are made, new integrations are added, major modules are introduced, or deployment infrastructure changes materially. Increment version, add change log entry, and circulate to all technical stakeholders for review.